



Date: __ / __ / __

Experiment 5

5. a. Implement Warshall's Algorithm to determine the transitive closure of a given directed graph represented as an adjacency matrix. The transitive closure of a graph determines all the vertices reachable from every other vertex in the graph, considering all possible paths. Warshall's Algorithm efficiently computes this closure using dynamic programming. The program should prompt the user to input the directed graph as an adjacency matrix and then apply Warshall's Algorithm to compute the transitive closure. Finally, print the resulting transitive closure matrix.

Type of Lab Activity: Exercise

Aim	Material	Methods	Answer
Given	Given	Given as Algorithm	Open

Algorithm

ALGORITHM Warshall($A[1..n, 1..n]$)

//Implements Warshall's algorithm for computing the transitive closure

//Input: The adjacency matrix A of a digraph with n vertices

//Output: The transitive closure of the digraph

$R^{(0)} \leftarrow A$

for $k \leftarrow 1$ **to** n **do**

for $i \leftarrow 1$ **to** n **do**

for $j \leftarrow 1$ **to** n **do**

$R^{(k)}[i, j] \leftarrow R^{(k-1)}[i, j] \text{ or } (R^{(k-1)}[i, k] \text{ and } R^{(k-1)}[k, j])$

return $R^{(n)}$

Program

(Only of faculty reference, should not be shared to students)

```
#include <stdio.h>
```

```
// Function to compute transitive closure using Warshall's algorithm
```

```
void transitiveClosure(int graph[][100], int n) {
```

```
    int i, j, k;
```

```
    // Adding vertices individually
```

```
    for (k = 0; k < n; k++) {
```

```
        // Fixing source vertices one by one
```

```
        for (i = 0; i < n; i++) {
```



```
// Fixing destination vertices one by one
for (j = 0; j < n; j++) {
    // If vertex k is on a path from i to j,
    // then mark it as reachable
    if (graph[i][k] && graph[k][j])
        graph[i][j] = 1;
}
}

// Displaying the transitive closure
printf("Transitive closure of the given graph is:\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        printf("%d ", graph[i][j]);
    }
    printf("\n");
}

int main() {
    int graph[100][100], n, i, j;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

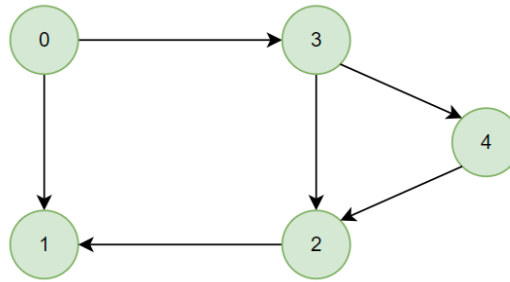
    // Input adjacency matrix
    printf("Enter the adjacency matrix (space-separated):\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Compute transitive closure
    transitiveClosure(graph, n);

    return 0;
}
```



Sample Output



Enter the number of vertices: 5

Enter the adjacency matrix (space-separated):

```
0 1 0 1 0
0 0 0 0 0
0 1 0 0 0
0 0 1 0 1
0 0 1 0 0
```

Transitive closure of the given graph is:

```
0 1 1 1 1
0 0 0 0 0
0 1 0 0 0
0 1 1 0 1
0 1 1 0 0
```